Extended Abstract

Motivation Math Reasoning is a foundational task for Large Language models (LLMs). Improving the performance of LLMs on Math Reasoning is important for creating more capable and aligned AI systems. In this project, we investigate the effects of applying failure-driven synthetic data augmentation and test-time inference (Best-of-N) techniques to enhance model performance on the Math Reasoning task. Specifically, we are interested in questions: how do these techniques affect performance (individually or combined), and under what conditions are these strategies more effective? *Side note*: As part of the default project, we also trained models for Instruction Following. Instruction Following is not the main focus of research, but we'll briefly document its implementation in the report for completeness.

Method We used Qwen 2.5 0.5B as the base model. For Math Reasoning, we adopted a multi-stage pipeline: (1) Baselines: Supervised fine-tuning (8) on the CognitiveBehavior dataset (7) and REINFORCE Leave-One-Out (1) fine-tuning on the Countdown dataset (9); (2) Best-of-N inference: At test time, sample N candidate outputs and select the one with the highest score as the final answer; (3) Failure-driven data augmentation: Perform failure analysis to identify failure patterns, generate synthetic examples that target those weaknesses, and retrain the model with the augmented dataset. The evaluation metric is a verifier score on Countdown task.

For Instruction Following, we used SFT on the SmolTalk dataset (2) and Direct Preference Optimization (11) on the UltraFeedback dataset (4). The evaluation metric is the win rate against a baseline model (selected by CS224R staff) on the Ultrafeedback task.

Implementation We developed a modular training pipeline supporting SFT, DPO and RLOO. For evaluation, we implemented inference scripts that leverage vLLM (12) as the inference engine. For Math Reasoning, we implemented synthetic data generation tools based on failure modes, and a flexible test-time inference module supporting Best-of-N inference. The score is computed using the same approach following TinyZero (10) which evaluates the format and the correctness of the response. For Instruction Following, we integrated with the Llama 3.1 Nemotron 70B Reward Model [1] to get the rewards for the responses.

Results Best-of-N inference consistently boosted performance. RLOO combined with Best-of-8 inference reached 0.74 (vs 0.44 for RLOO). Applying failure-driven synthetic data augmentation on top of RLOO led to further gains. The augmented model with Best-of-8 inference achieved the highest score of 0.83. We evaluated two synthetic datasets: one targeting basic failure modes (Aug-1), and another including examples with multiplication/division (Aug-2). Both improved over RLOO, though gains were more obvisous in simpler cases. These results demonstrate the complementary effects of test-time inference and targeted augmentation in enhancing LLM reasoning performance.

Discussion The results highlight the effectiveness of combining inference-time and data augmentation strategies for math reasoning. Best-of-N inference improves performance by leveraging the stochastic nature of generation, often recovering correct answers missed by single-sample decoding. Meanwhile, failure-driven synthetic augmentation strengthens model robustness, particularly on targeted weaknesses such as large numbers. Its impact is amplified at higher N values, suggesting that augmentation adds latent capabilities which Best-of-N helps surface. While augmentation showed limited gains on more complex arithmetic (e.g., multiplication/division), this points to opportunities for more expressive or diverse synthetic data.

Conclusion Our experiments show that combining reinforcement-based fine-tuning with test-time inference and targeted data augmentation yields substantial gains in math reasoning tasks. Best-of-N inference consistently improves accuracy, and failure-driven synthetic data provides further improvements when integrated carefully. Together, these modular techniques raise the verifier score from 0.44 (RLOO) to 0.83 (RLOO + augmentation + Best-of-8), demonstrating the effectiveness of the best-of-N and failure-driven synthetic data augmentation strategies for LLM enhancement.

[1] `https://huggingface.co/nvidia/Llama-3.1-Nemotron-70B-Reward`

Failure-Driven Synthetic Data Augmentation and Best-of-N Inference on LLM Math Reasoning

Fan Diao
Stanford University
fdiao@stanford.edu

Abstract

Improving the mathematical reasoning capabilities of large language models (LLMs) is essential for building more capable and aligned AI systems. In this project, we investigate the effects of failure-driven synthetic data augmentation test-time inference methods such as Best-of-N sampling (individually or combined). We apply these strategies to the Countdown task, using the CognitiveBehavior and Countdown datasets. Our results demonstrate that Best-of-N inference significantly boost math reasoning performance, with Best-of-8 reaching a top verifier score of 0.74 on Countdown. Failure-driven data augmentation provides additional gains when carefully integrated, reaching the highest score of 0.83 when combined with Best-of-8. These findings highlight the value of combining targeted synthetic augmenation and inference strategies to enhance LLM performance in a modular fashion.

1 Introduction

Large language models (LLMs) have achieved remarkable progress in general-purpose tasks, yet significant challenges remain in structured domains such as math reasoning. Improving model performance in this area is critical for alignment, interpretability, and robust downstream applications.

Post-training techniques such as Supervised Fine-Tuning (SFT), reward optimization, and reinforcement learning have emerged as effective methods to specialize general LLMs for downstream tasks. More recently, inference-time strategies like Best-of-N sampling and dataset-level augmentation methods have also shown promise.

In this project, we evaluate how failure-driven synthetic data augmentation and best-of-N inference can individually or jointly enhance LLM performance on the Math Reasoning task, where the model must solve arithmetic puzzles in a multi-step format.

We established staged pipelines that combine SFT and RLOO for baseline training, and we explored Best-of-N inference and failure-driven synthetic data augmentation for further improvements. Our goal is to understand how the combinations of these techniques improve performance, and under what conditions they yield more effective performance gains.

Side note: as part of the default project, we also trained models for Instruction Following. Instruction Following is not the main focus of research, but we'll briefly document the experiments for Instruction Following in the report for completeness.

2 Related Work

Recent advances have significantly improved LLM reasoning through supervised fine-tuning and reinforcement learning. (5) demonstrated that large-scale reinforcement learning (RL) alone (DeepSeek-

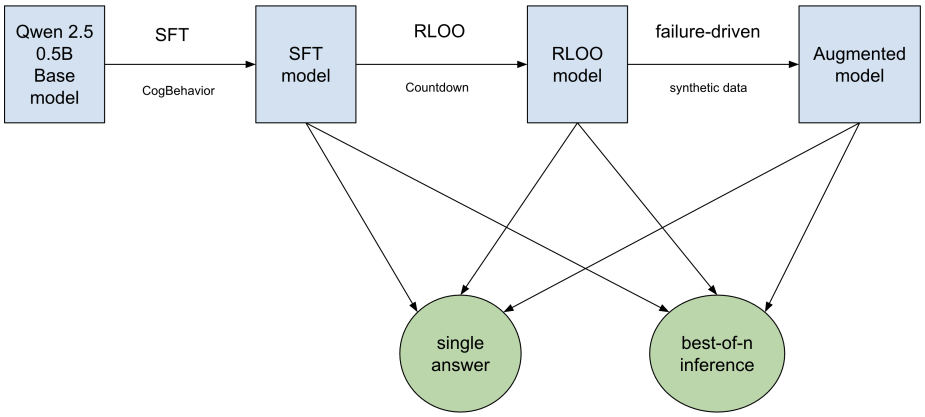

Figure 1: Summary of the stages for Math Reasoning.

Figure 2: Summary of the stages for Instruction Following.

R1-Zero) could induce strong reasoning behaviors but exhibited poor readability and mixed-language generation. DeepSeek-R1 subsequently combined cold-start supervised fine-tuning with RL, achieving strong reasoning while improving output fluency. Similarly, our project adopts the multi-stage SFT and RL training. However, we will take a more lightweight approach by improving SFT and RL through failure-driven synthetic data, better suited for resource-constrained academic settings.

Failure-driven data augmentation has emerged as a promising apporach to systematically fix model weaknesses. (6) introduced counterexample-guided data augmentation, enhancing models by augmenting on observed failure cases. Inspired by this, we will design synthetic data generation aligned with model-specific reasoning errors.

Best-of-N inference is a widely used technique to improve the quality of generated outputs by sampling multiple candidates and selecting the one with the highest reward or score. While previous work like Wang et al. (13) showed the effectiveness of self-consistency via majority voting in reasoning tasks, Deng et al. (3) take this further by proposing *inference-aware fine-tuning*—a method that explicitly optimizes the model for best-of-N sampling during training. Their approach improves the alignment between training and inference objectives, especially for tasks where reward functions are sparse or discontinuous. In contrast, our work focuses on applying best-of-N inference as a plug-and-play technique on top of standard or reinforcement-trained models, showing that even without inference-aware training, best-of-N yields substantial performance gains on math reasoning tasks.

In summary, while prior works have separately leveraged failure-driven augmentation and best-of-N inference, our project uniquely integrates these strategies into a unified, modular pipeline across training stages for resource-efficient math reasoning enhancement.

3 Method

3.1 Math Reasoning

Figure 1 illustrates our approach. It includes multiple stages training and inference optimizations.

- **SFT (Baseline #1)**: Supervised Fine Tuning on CognitiveBehavior dataset.
- **RLOO (Baseline #2)**: REINFORCE Leave One-Out training on Countdown dataset.
- **Best-of-N test-time inference (Extension #1)**
  - Sample n candidate answers, pick the one with the highest score as the final answer.
- **Failure-driven synthetic data augmentation (Extension #2)**
  - Do failure analysis to identify failure patterns (count of numbers, number ranges, operations, etc).
  - Generate synthetic puzzles matching the failure patterns.
  - Run additional training with synthetic puzzles.

3.1.1 Supervised Fine-Tuning

Using QWen 2.5 0.5B model as a base model, we performed Supervised Fine-Tuning on the CognitiveBehavior dataset. Supervised Fine-Tuning trains the model to predict the next token in a way that mimics the training dataset. In essence, it aims to learn the expert distribution from queries x and completions y. The supervised learning objective is as below.

$$\max_{\theta} \mathbb{E}_{x,y \in D} \left[\sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}) \right] \tag{1}$$

3.1.2 REINFORCE Leave One-Out

Based on the SFT model from the first stage, we then optimize the model using RLOO (1), an online policy-gradient algorithm based on REINFORCE with a baseline to reduce variance of samples. The RLOO objective is given by:

$$\frac{1}{k} \sum_{i=1}^{k} \left[R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x) \right] \nabla \log \pi_\theta(y_{(i)} \mid x) \quad \text{for } y_{(1)}, \ldots, y_{(k)} \overset{i.i.d.}{\sim} \pi_\theta(\cdot \mid x) \tag{2}$$

Here, $R(y, x)$ is the reward function for output y given input x, and π_θ is the current policy being optimized. The leave-one-out baseline (i.e., the average reward from all other samples) serves to reduce the variance of the policy gradient update for each sample $y_{(i)}$.

As we focus on the Countdown task, we compute the reward in the same way as TinyZero (10), by evaluating the reformat and the correctness of the answer.[2]

3.1.3 Extension #1: Best-of-N Test-Time Inference

Best-of-N is a simple yet effective test-time inference strategy used to improve the final prediction quality of a generative language model. Rather than relying on a single stochastic generation, we let the model generate multiple candidate outputs, and select the one with the highest score as the final answer.

Given an input x, the model π_θ is sampled N times to produce candidate outputs:

$$y_1, y_2, \ldots, y_N \sim \pi_\theta(\cdot \mid x) \tag{3}$$

Each output y_i is then evaluated using a reward function $R(y_i, x)$.

The final output is selected as:

$$y^* = \arg \max_{i \in \{1,\ldots,N\}} R(y_i, x) \tag{4}$$

In our setting, $R(y, x)$ is a deterministic function that measures the format and the correctness of the answer.

[2] `https://github.com/kanishkg/cognitive-behaviors/blob/main/verl/utils/reward_score/countdown.py`

3.1.4 Extension #2: Failure-Driven Synthetic Data Augmentation

Failure Analysis After evaluating the models trained with SFT and RLOO, we perform failure analysis to identify the failure patterns. We category the test samples based on the following factors:

- Count of numbers: How many numbers are given to reach the target? For example, 3, 4, 5 and 6.
- Range of numbers: Do the numbers fall into the range of $[0, 100)$, or $[100, 1000)$, or $[1000, 10000)$, etc?
- Operations involved: Does the expected equation involve only addition and subtraction, or does it involve multiplication or division?

Then we can analyze for each category of samples, the success rate and failure rate. This helps to identify the weakness of the models.

Synthetic Data Generation Based on the failure modes identified from the above step, we would like to generate synthetic puzzles with expected responses, so that we can apply additional SFT. To generate synthetic puzzles, we do the following:

- Define the attributes for the category of samples to generate: the count of numbers N, the range of numbers $[S, E)$, whether the expected equation involves multiplication has_mul, whether the expected equation involves division has_div.
- Randomly generate N numbers within the range $[S, E)$.
- Based on the operations, randomly generate an equation and corresponding targets.

With the synthetic puzzles generated, we then integrated with a stronger LLM (*claude-3-5-sonnet-20241022*) to generate the expected response. We adapt the LLM prompt from Cognitive Behaviors (7). [3]. Although *Claude 3.5* is one of the strongest LLM as of now, it doesn't always return the correct answer for every puzzle. Thus, we introduce an additional filtering step that filters only samples for which *Claude 3.5* returns a correct answer.

The synthetic puzzles and responses are formatted in the same way as the CognitiveBehavior dataset.

Additional Training After generating the synthetic dataset, we then perform additional SFT training based on the RLOO model. The objective is also the same as Equation (1).

3.2 Instruction Following

Figure 2 illustrates our approach. It includes two-stage training: SFT and DPO.

3.2.1 Supervised Fine-Tuning

Similarly to Math Reasoning, we used QWen 2.5 0.5B model as a base model, we performed Supervised Fine-Tuning on the SmolTalk dataset. The objective is also the same as Equation (1).

3.2.2 Direct Preference Optimization

Using the SFT model trained from above stage as the base model, we performed direct preference optimization (11) on the ultrafeedback data set. Each sample in the dataset includes a prompt x, a winning response y_w and a losing response y_l. With DPO, we optimize for the following loss.

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right] \tag{5}$$

where x is the prompt, y_w is the winning response, y_l is the losing response, π_θ is the policy that's being trained, and π_{ref} is the reference policy.

[3] `https://github.com/kanishkg/cognitive-behaviors?tab=readme-ov-file`

4 Experimental Setup

4.1 Math Reasoning

Experiment Setting SFT and RLOO are the baselines. For supervised fine-tuning on the CognitiveBehavior dataset, we used Adam as the optimizer with a learning rate of 2×10^{-5}, batch size 16 (after gradient accumulation), and trained for 5 epochs. We truncated the input at 1280 tokens. The truncation was done so that over 95%of samples are below the threshold without hitting GPU OOM. For RLOO on the Countdown dataset, we trained with 1000 samples (we didn't use the full dataset due to time constraint as RLOO training takes time). We chosen K = 12. We used Adam as the optimizer with a learning rate of 1×10^{-6}, batch size 8 (after gradient accumulation).

For failure-driven synthetic data augmentation, we experimented with two types of synthetic augmentation. Both are based on the RLOO model.

- **Synthetic #1**: Perform additional SFT training using synthetic set #1 mixed with original data. The synthetic set #1 has 318 samples. These samples match failure patterns on the range of numbers and the count of numbers. They don't involve multiplication or division operation.
- **Synthetic #2**: Perform additional SFT training using synthetic set #2 mixed with original data. The synthetic set #2 has 518 samples. On top of the 318 samples in synthetic set #1, the synthetic set #2 adds 200 more samples that involve multiplication and division operations.

For best-of-N inference, we experimented with $N = 1, 2, 4, 8$.

Evaluation Metric We evaluate the average verifier-based score on the Countdown questions: given a target number and a list of integers, the model tries to reach the target using basic operations. The score is computed using the same approach following TinyZero ((10)) which evaluates the format and the correctness of the response.

4.2 Instruction Following

Experiment Setting SFT is the baseline. For supervised fine-tuning on the SmolTalk dataset (459K train / 1K validation), we used Adam as the optimizer with a learning rate of 5×10^{-5}, batch size 128 (after gradient accumulation), and trained for 2 epochs. We truncated the input at 1280 tokens. For DPO on the Ultrafeedback dataset (60K train / 1K val), we used RMSProp as the optimizer with a learning rate of 1×10^{-6}, batch size 128 (after gradient accumulation), and also trained for 2 epochs.

Evaluation Metric Given the held-out Ultrafeedback prompts provided by the staff, we evaluate the models by submitting the responses to the leaderboard which computes the win rate against a baseline model (selected by the staff). The win rate is defined as the ratio where our trained models return a higher-reward response than the baseline model.

5 Results

5.1 Math Reasoning

5.1.1 Quantitative Evaluation

Table 1 shows the performance comparison of various models with various best-of-N inference strategies. Based on the results, the findings are:

- SFT achieved an average score of 0.36. RLOO reached an average score of 0.44, i.e. 0.09 higher than SFT.
- For all models, best-of-N inference improves the performance significantly. As the number N increases, the performance also increases. However, the increasing rate slows down as N increases. For RLOO model, best-of-8 achieved an average score of 0.74.

Method		Avg Score	Score Distribution		
Best of N	**Model**		**1**	**0.1**	**0**
N = 1	SFT (Baseline)	0.36	303	564	133
	RLOO (Baseline)	0.44	363	431	206
	Aug-1	**0.45**	407	422	171
	Aug-2	**0.45**	405	427	168
N = 2	SFT	0.51	457	516	27
	RLOO	0.6	565	386	49
	Aug-1	0.62	583	360	57
	Aug-2	**0.64**	603	356	41
N = 4	RLOO	0.71	674	319	1
	Aug-1	0.73	703	284	13
	Aug-2	**0.75**	719	261	20
N = 8	RLOO	0.74	711	288	1
	Aug-1	**0.83**	812	187	1
	Aug-2	0.82	801	196	3

Table 1: Performance comparisons of various models with various best-of-N inference strategies. The 'Avg Score' column is the average score over the 1000 held-out Countdown samples provided by staff. The 'Score Distribution' columns are the count of samples for each score, where 1 means format and equation are both correct, 0.1 means format is correct but equation is wrong, 0 means format is not correct. Aug-1 model means the checkpoint from Synthetic #1 training. Similarly, Aug-2 model is the checkpoints from Synthetic #2 training.

- With failure-driven synthetic data augmentation, Aug-1 and Aug-2 models (from Synthetic #1, #2) improve the performance on top of RLOO model. Compred to the RLOO model, the performance gain ranges from 0.01 to 0.09, varing depending on the N chosen in the best-of-N inference. The larger N is, the larger the performance gain is. When N = 8, the performance gain is the largest. Aug-1 model combined with best-of-8 reached the highest overall score of 0.83. If we look at the score distribution, we observe that Aug-1 and Aug-2 increased the number of full-score samples by 18 to 101. Full-score examples mean that both the format and equations are correct.

- Although Aug-2 model trains on more synthetic samples involving multiplication or division than Aug-1, the performances of Aug-1 and Aug-2 models don't show a large difference.

Failure Pattern Analysis: To further drill down to see whether the failure-driven synthetic data augmentation helps to address some of the failure modes, table 2 shows the comparison of the failure rates for a few selected categories of samples for different models with best-of-1 inference. We can see that the augmentation does help to decrease the failure rate of some patterns such as when given 3 or 4 large numbers. However, it doesn't always help. For example, when given 5 or 6 numbers, or when the operations include multiplication or division, the gain is small. The below section 5.1.2 show some examples of where the synthetic data augmentation helps.

5.1.2 Qualitative Analysis

Figure 3 shows some example questions and corresponding answers from various models with various best-of-N inference strategies. Figure 3 (a) shows an example where RLOO and Aug-1 model both generate correct answers regardless of using best-of-1 or best-of-8 inference. Figure 3 (b) shows an example where Aug-1 outperforms RLOO. Figure 3 (c) shows an example where best-of-8 inference generated the correct answers while best-of-1 did not. Firgure 3 (d) is an interesting example where the question gives 4 numbers in the range of $[100000, 1000000)$. In this example, RLOO with best-of-8 didn't produce the correct answer, but Aug-1 was able to. This highlights that a case where the failure-driven synthetic data augmentation helps.

Count of Nums	Category Range of Nums	Mul	Div	Method Failure Rate RLOO	Aug-1	Aug-2
3	[1000, 10000)	False	False	0.42	**0.36**	0.34
	[10000, 100000)	False	False	0.53	0.51	**0.41**
	[0, 100)	True	False	0.8	0.8	**0.73**
	[0, 100)	False	True	0.9	**0.9**	1.0
4	[1000, 10000)	False	False	0.79	0.76	**0.4**
	[10000, 100000)	False	False	0.91	0.76	**0.55**
	[0, 100)	True	False	1.0	1.0	1.0
	[0, 100)	False	True	**0.67**	1.0	1.0
5	[0, 100)	False	False	0.91	**0.86**	0.91
	[100, 1000)	False	False	1.0	**0.86**	0.91
6	[0, 100)	False	False	1.0	1.0	**0.86**
	[100, 1000)	False	False	1.0	1.0	1.0

Table 2: Comparisons of failure rates for a few selected categories of samples. A failure is when the model is unable to generate the correct equation.

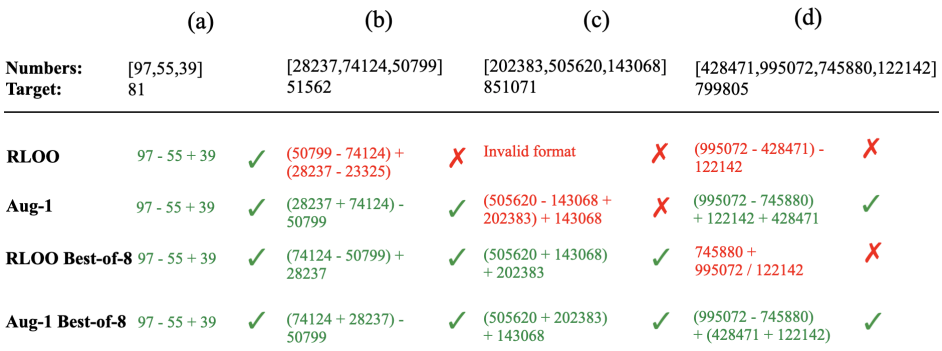

Figure 3: Visualization of results on Countdown.

5.2 Instruction Following

5.2.1 Quantitative Evaluation

The results are reported in Table 3 and 4. Table 3 are the scores based on the V1 Leaderboard (the one for milestone). Table 4 are the scores based on the V2 Leaderboard (the one for final submission). Based on the scores in V1 Leaderboard, we can see DPO outperforms SFT by 0.08. The baseline model in V2 Leaderboard is stronger than V1 Leaderboard, hence the difference in the scores for DPO.

Model	V1 Leaderboard Score
SFT	0.83
DPO	**0.91**

Table 3: Performance on V1 Leaderboard.

Model	V2 Leaderboard Score
SFT	N/A
DPO	**0.21**

Table 4: Performance on V2 Leaderboard.

5.2.2 Qualitative Analysis

Figure 3 shows an example prompt and the responses from SFT and DPO. The SFT response gets a reward of -25.375. The DPO response gets a reward of -13.375. The DPO response outperforms the SFT response across all key dimensions: structure, completeness and technical specificity. It delivers a well-organized, actionable, and thorough workflow for front-end development.

Prompt: What is the typical workflow for a front-end developer to design and implement a responsive user interface for a web application?

SFT Response (reward = -25.375):

The typical workflow for a front-end developer to design and implement a responsive user interface (UI) for a web application involves several stages. It begins with creating wireframes or mockups to visualize the layout and structure of the application. This involves sketching out a rough outline of the user interface, including the navigation, buttons, forms, and other interactive elements.

Next, the developer creates a prototype or a simple version of the application to test and refine the design. This allows them to identify any issues with the layout or navigation and make necessary adjustments before moving on to the actual implementation.

After refining the design, the developer proceeds to write the necessary CSS and JavaScript code to implement the design. This may involve using frameworks or libraries that provide pre-built components or templates for common UI elements.

Once the code is ready, the developer tests the application to ensure that it functions as intended, including the responsiveness and accessibility of the UI. This involves testing for different screen sizes, zoom levels, and other interactions that may be expected.

Finally, the developer makes any necessary adjustments and continues to test and refine the UI until it meets the desired level of responsiveness and usability. Throughout the process, the developer should also consider the usability, accessibility, and performance of the UI to ensure that it is effective and enjoyable for the intended users.

DPO Response (reward = -13.375):

Here's a typical workflow for a front-end developer to design and implement a responsive user interface for a web application:

1. **Define the project requirements**: The developer should gather information about the application, including its target audience, features, and performance needs. This involves understanding the problem the application aims to solve, the desired user experience, and any specific requirements or constraints.

2. **Create wireframes and mockups**: Using tools like Figma, Sketch, or Adobe XD, the developer creates rough, visual representations of the application's layout and user flow. This helps identify potential issues, such as navigation, layout, and interaction problems.

… (skipped for rendering)

7. **Refine and deploy**: Once the design is finalized, the developer refines it further and deploys the application to a production environment. This involves setting up hosting, configuring the necessary dependencies, and ensuring that the application is compatible with various devices and screen sizes.

8. **Monitor and maintain**: After deployment, the developer continues to monitor the application's performance, user experience, and technical requirements. They make adjustments and improvements as needed to ensure the application remains responsive and meets the desired performance standards.

By following this workflow, a front-end developer can create a responsive user interface for their web application that provides a seamless and intuitive experience for users across various devices and screen sizes.

Figure 4: Visualization of results on Ultrafeedback.

6 Discussion

6.1 Impact of Best-of-N Inference on Countdown

For all models, best-of-N inference improves the performance significantly. As the number N increases, the performance also increases. However, the increasing rate slows down as N increases. We infer that the performance gain by best-of-N inference is because the model generation process is stochastic. It may not produce the exact correct answer each time, but can often include at least one correct answer when N is large enough. Best-of-N inference gives the model multiple chances to generate candidate answers and pick the one with the highest score.

6.2 Impact of Failure-Driven Synthetic Data Augmentation on Countdown

Based on the results, we see that the failure-driven synthetic data augmentation helps to improve the performance in some cases. For example, when given 3 or 4 large numbers. However, for questions that are given 5 or 6 numbers, or involve multiplication or division, the gain is very small. This is likely because the latter types of questions are harder and they may require synthetic data of higher quality and larger size.

On the other hand, the overall performance gain is not as large as best-of-N on the Countdown task. We infer that synthetic data augmentation on Countdown requires carefully curated data with sufficient amount to see large gains.

6.3 Combined Best-of-N and Synethetic Data Augmentation

We have another interesting observation that, as N increases, the augmented model outperforms the RLOO model by a larger amount. We infer that indicates the synthetic augmentation adds more potential to the model and best-of-N inference allows the model to uncover its potential by generating more candidate outputs as N gets bigger.

7 Conclusion

We investigated Best-of-N test-time inference and failure-driven synthetic data augmentation for improving math reasoning in LLMs. Best-of-N consistently boosted performance, with the highest

score (0.83) achieved using synthetic augmentation and $N = 8$ inference. Augmented models showed greater gains as N increased, highlighting synergy between the two techniques.

While synthetic data helped address specific failure modes, its impact was uneven—particularly on more complex operations like multiplication and division.

Future Work: Extensions could include applying these techniques to other reasoning tasks, scaling to larger models, or improving synthetic data coverage and quality.

8 Team Contributions

Fan Diao (Alone):

- All implementation and experiments
- Project proposal, milestone, poster and report preparation

Changes from Proposal

- Added best-of-N test-time inference experiments.
- Removed curriculum learning experiments due to time constraints.

References

[1] A. Ahmadian, C. Cremer, M. Gallé, M. Fadaee, J. Kreutzer, O. Pietquin, A. Üstün, and S. Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024.

[2] L. B. Allal, A. Lozhkov, E. Bakouch, G. M. Blázquez, G. Penedo, L. Tunstall, A. Marafioti, H. Kydlíček, A. P. Lajarín, V. Srivastav, J. Lochner, C. Fahlgren, X.-S. Nguyen, C. Fourrier, B. Burtenshaw, H. Larcher, H. Zhao, C. Zakka, M. Morlon, C. Raffel, L. von Werra, and T. Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025.

[3] Y. Chow, G. Tennenholtz, I. Gur, V. Zhuang, B. Dai, S. Thiagarajan, C. Boutilier, R. Agarwal, A. Kumar, and A. Faust. Inference-aware fine-tuning for best-of-n sampling in large language models, 2024.

[4] G. Cui, L. Yuan, N. Ding, G. Yao, W. Zhu, Y. Ni, G. Xie, Z. Liu, and M. Sun. Ultrafeedback: Boosting language models with high-quality feedback, 2023.

[5] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.

[6] T. Dreossi, S. Ghosh, X. Yue, K. Keutzer, A. Sangiovanni-Vincentelli, and S. A. Seshia. Counterexample-guided data augmentation, 2018.

[7] K. Gandhi, A. Chakravarthy, A. Singh, N. Lile, and N. D. Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars, 2025.

[8] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback, 2022.

[9] J. Pan. Countdown tasks 3 to 4. https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4, 2024. Accessed: 2025-06-08.

[10] J. Pan. Tinyzero. https://github.com/Jiayi-Pan/TinyZero, 2024. Accessed: 2025-06-08.

[11] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.

[12] vLLM Team. vllm: A high-throughput and memory-efficient inference engine for llms. https://docs.vllm.ai/en/latest/, 2023. Accessed: 2025-06-08.

[13] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models, 2023.